

# Почему EAGLE-3 на H200 ускоряет меньше, чем в статье

Профиль кода авторов: LLaMA-3.1-8B-Instruct, официальный драфт, одна H200

Смена по воспроизведению EAGLE-3

9 октября 2026

## Коротко

1.  **$\tau$  совпадает со статьёй** (6,25 против 6,23 при  $T=0$ ), а ускорение ниже (3,38× против 4,44×). Значит, вся разница — в стоимости одного цикла EAGLE, выраженной в шагах обычной генерации: у нас 1,64, у авторов  $\approx 1,40$ .
2. **Цикл дороже уже по чистой работе GPU**: 13,4 мс против 8,2 мс у обычного шага. Драфт — 885 микроскопических ядер по  $\approx 4$  мкс (3,6 мс), проверка дерева из 60 токенов — 9,7 мс.
3. **Мелкие ядра почти не ускоряются на быстром GPU, а обычный шаг на H200 очень быстрый** (веса 16 ГБ читаются за 3,3 мс). Если подставить в те же замеры память уровня A100, цикл выходит  $\approx 1,40$  шага и ускорение  $\approx 4,5\times$  — как в статье.
4. **CPU не причина** (AMD EPYC 9555, Zen 5): накладные расходы на запуск ядер одинаково замедляют и обычный шаг, и цикл EAGLE.
5. **Вывод**: разница — свойство железа, а не ошибка воспроизведения. Окончательно подтверждает замер на A100 (GPU для Table 1 в статье не указан).

## Как мерили

Код авторов (SafeAllLab/EAGLE, коммит cb7e084) без изменений, дерево как в статье EAGLE-3 (60 токенов, глубина 8, top-k 10),  $T=0$ , fp16; MT-bench, первые ходы 5 вопросов. Скрипт `profile_eagle.py` оборачивает фазы цикла снаружи: время каждой фазы — между синхронизациями CUDA, а `torch.profiler` даёт длительность каждого CUDA-ядра. **GPU-работа** — сумма длительностей ядер, запущенных из фазы; на неё профилировщик не влияет (в отличие от общего времени, которое он растягивает, нагружая CPU).

Ускорение =  $\tau$  / (стоимость цикла в шагах обычной генерации). При одинаковом  $\tau$  ускорение определяется только тем, во сколько обычных шагов обходится цикл.

## Из чего состоит цикл

|                                              | GPU-работа, мс | Ядер         | Общее время, мс |
|----------------------------------------------|----------------|--------------|-----------------|
| Шаг обычной генерации                        | 8,2            | 1 766        | 12,7            |
| <b>Цикл EAGLE</b>                            | <b>13,4</b>    | <b>2 663</b> | <b>21,3</b>     |
| проверка дерева большой моделью (60 токенов) | 9,7            | 1 758        | 13,6            |
| драфт (8 шагов дерева)                       | 3,6            | 885          | 6,8             |
| выбор ветки и обновление кэша                | 0,1            | 20           | 0,4             |
| Цикл / обычный шаг                           | <b>1,64</b>    | <b>1,51</b>  | <b>1,67</b>     |

- **Отношение по чистой работе GPU (1,64) почти равно отношению по общему времени (1,67).** Значит, простой GPU в ожидании CPU добавляет примерно одинаковую долю и к обычному шагу, и к циклу, и разницу со статьёй не объясняет.
- **Проверка дерева из 60 токенов стоит на 19 % дороже обычного шага по работе GPU (9,7 против 8,2 мс)** — ожидаемо: та же модель, но 60 позиций вместо одной.
- **Драфт — главная добавка:** 3,6 мс GPU-работы, 0,44 обычного шага. При этом его 885 ядер в среднем по  $\approx 4$  мкс: время уходит на задержку запуска и исполнения каждого маленького ядра, а не на объём вычислений.

## Почему у авторов цикл дешевле

Обычный шаг 8B-модели при генерации по одному токену ограничен чтением весов из памяти: 16 ГБ за шаг. На H200 (4,8 ТБ/с) это 3,3 мс — поэтому обычный шаг очень быстрый. Драфт и накладные расходы на мелкие ядра от пропускной способности памяти почти не зависят. Чем быстрее память GPU, тем дешевле обычный шаг, а цена драфта остаётся — его доля в цикле растёт, ускорение падает.

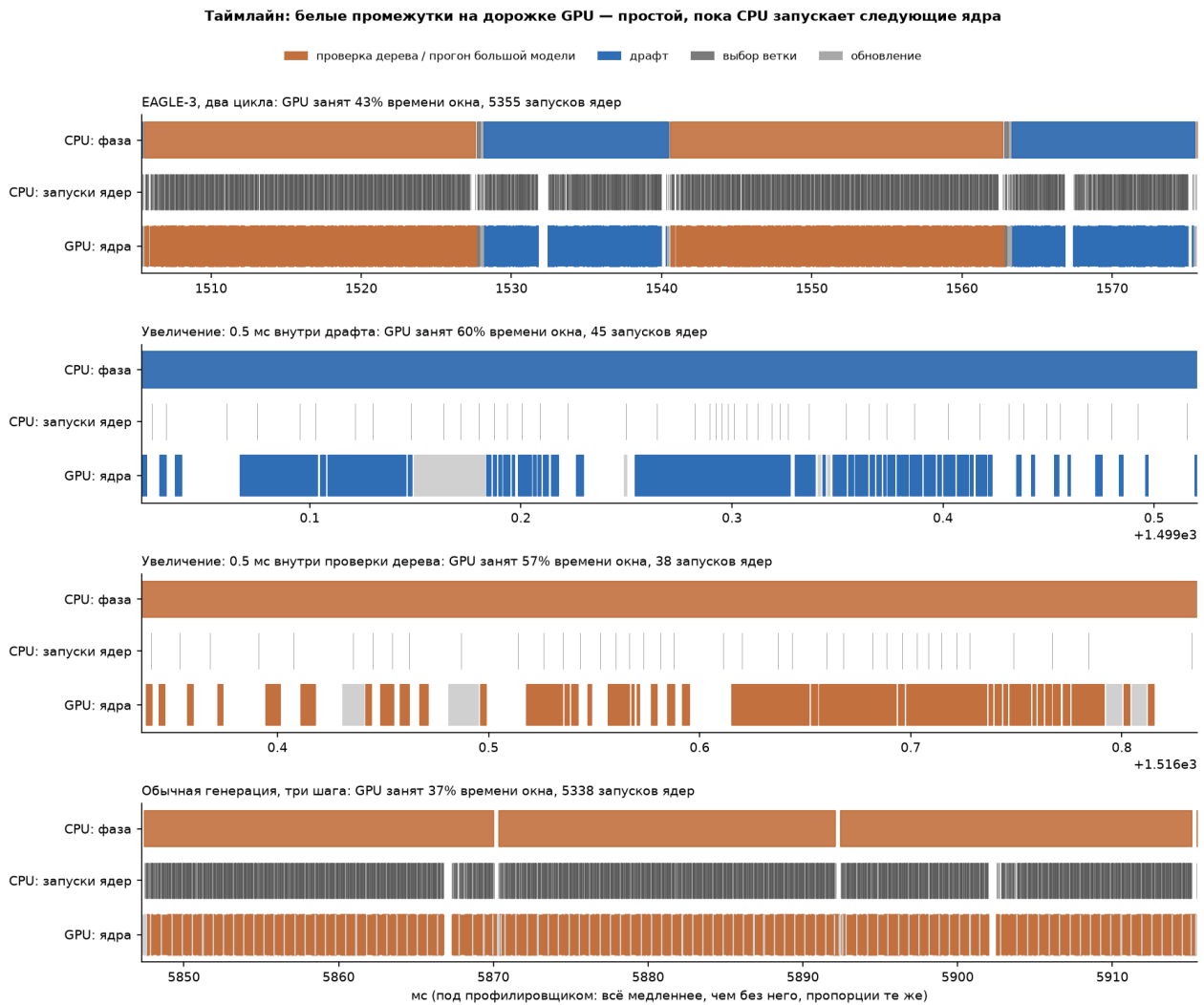
Проверка: в тех же замерах заменяем только время чтения весов (обычный шаг и проверка дерева дорожают на эту разницу, драфт не меняется).

| GPU           | Чтение весов | Обычный шаг, мс | Цикл, мс | Цикл / шаг     | Ускорение при $t = 6,25$ |
|---------------|--------------|-----------------|----------|----------------|--------------------------|
| H200 (замер)  | 3,3 мс       | 8,2             | 13,4     | 1,64           | 3,4–3,8× (замер)         |
| H100 (оценка) | 4,8 мс       | 9,6             | 14,9     | $\approx 1,55$ | $\approx 4,0\times$      |
| A100 (оценка) | 8,0 мс       | 12,9            | 18,1     | $\approx 1,40$ | $\approx 4,5\times$      |
| Статья        | —            | —               | —        | $\approx 1,40$ | 4,44×                    |

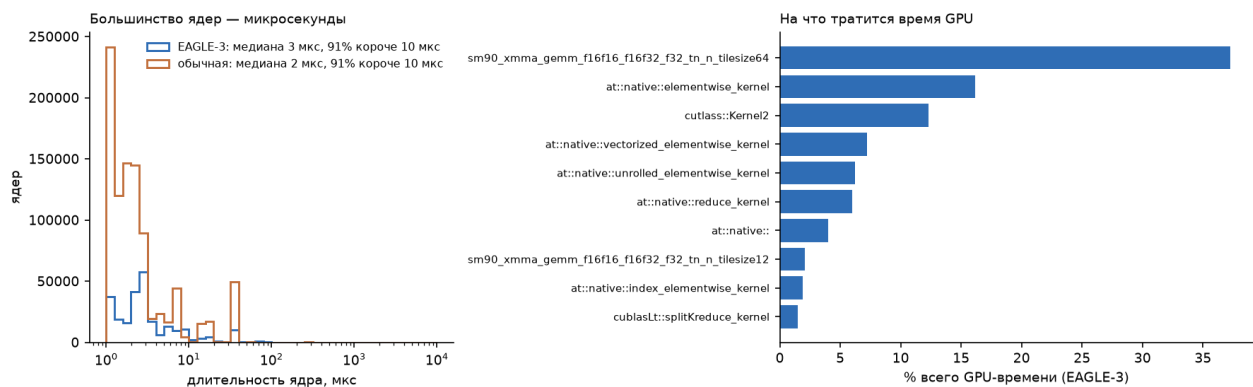
- **Цифры статьи совпадают с оценкой для GPU с памятью уровня A100.** GPU для Table 1 в статье не указан, поэтому это сильный косвенный довод, а не доказательство. Модель грубая: она учитывает только чтение весов.
- **CPU не причина:** AMD EPYC 9555 (Zen 5, до 4,4 ГГц) — быстрый серверный процессор, а простой GPU одинаково влияет на обычный шаг и цикл (см. первую таблицу).

## Что из этого следует

- **Воспроизведение корректно:** драфт, дерево и протокол дают  $t$  статьи; разница в ускорении — от железа.



**Рис. 1:** Таймлайн: два цикла EAGLE (проверка дерева — оранжевым, драфт — синим), увеличенные окна по 0,5 мс внутри драфта и внутри проверки, и три шага обычной генерации. Верхняя дорожка — фаза на CPU, средняя — запуски ядер, нижняя — ядра на GPU. Белые промежутки на дорожке GPU — простой. Под профилировщиком CPU медленнее, поэтому простоя здесь больше, чем без него (без профилировщика GPU занят  $\approx 65\%$  и в обычном шаге, и в цикле).



**Рис. 2:** Слева: 91 % ядер короче 10 мкс (медиана 2–3 мкс) и в обычной генерации, и в EAGLE. Время таких ядер задаётся задержкой запуска и исполнения и почти не уменьшается на более быстром GPU. Справа: на что тратится время GPU в EAGLE — GEMM-ядра и мелкие поэлементные операции.

- **На быстрых GPU ускорение EAGLE в исследовательском коде ниже.** Чтобы получить больше на H200, нужно сократить число мелких ядер драфта и проверки — слияние операций, CUDA graphs. Именно это делает SGLang; замер в нём (задача 3b) сейчас идёт.
- **Прямая проверка гипотезы** — тот же профиль на A100 (около 15 минут).

Код и сырые числа: [reproduction/](#) (code/inference\_speed/profile\_eagle.py, plot\_profile.py, results/profile\_3a.md).